



Projeto e Análise de Algoritmos

Algoritmos de Grafos

André Tavares da Silva
andre.silva@udesc.br

Grafos

Grafos possuem dois elementos principais:

- Vértices – Vertex (V)
- Arestas – Edges (E)

Grafos costumam ser representados de duas formas, que utilizam uma quantidade diferente de **espaço**:

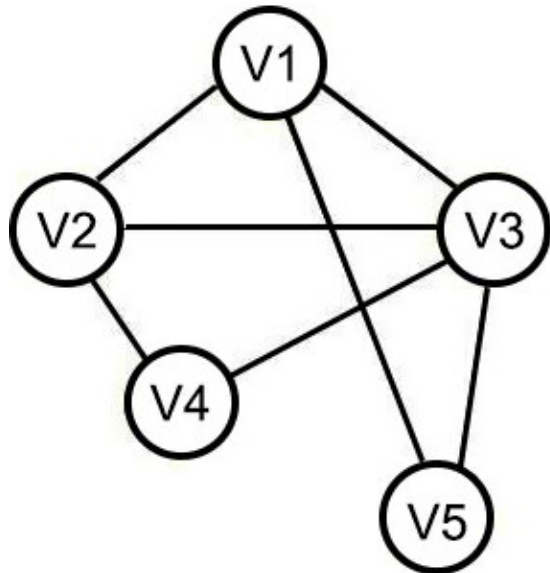
- Matriz de Adjacência $\Theta(V^2)$
- Lista de Adjacência $\Theta(V+E)$

* Considere apenas o espaço criado pelo algoritmo

Grafos

Matriz de Adjacência

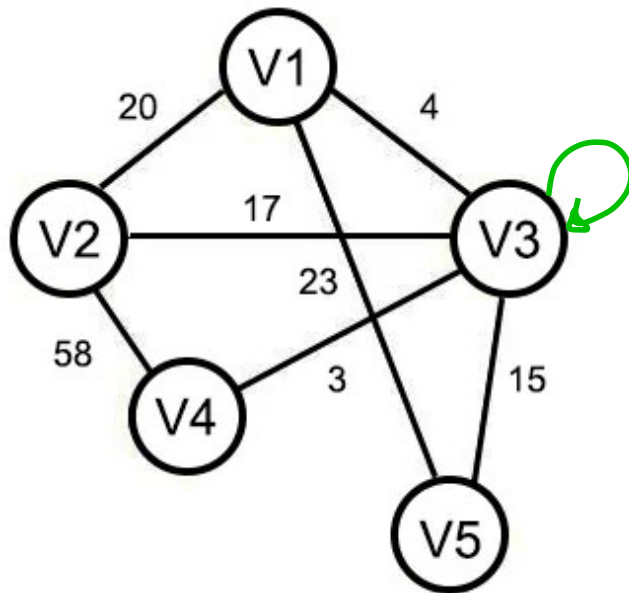
$\Theta(V^2)$



	V1	V2	V3	V4	V5
V1	0	1	1	0	1
V2	1	0	1	1	0
V3	1	1	0	1	1
V4	0	1	1	0	0
V5	1	0	1	0	0

Grafos

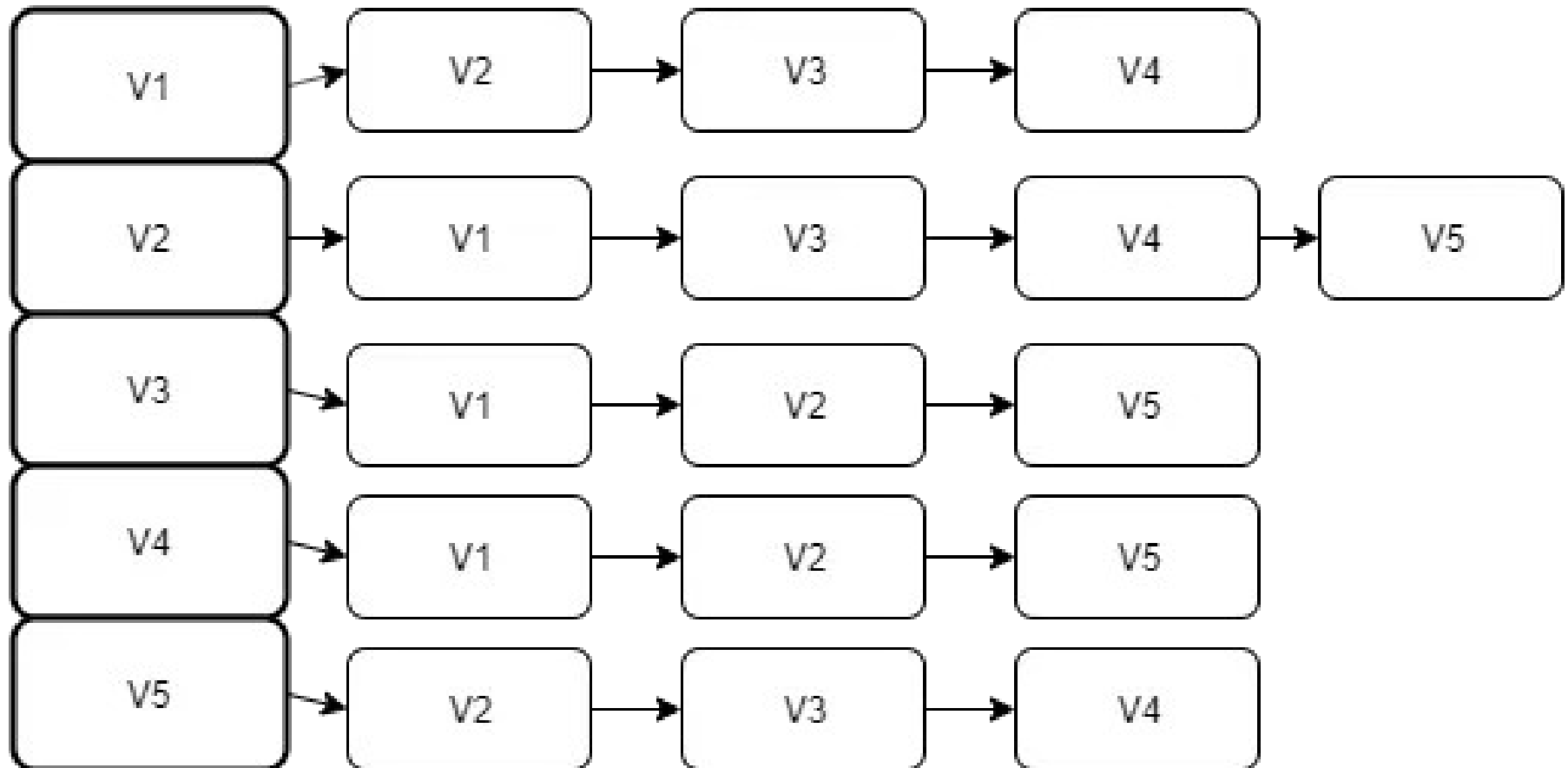
Matriz de Adjacência - grafo ponderado $\Theta(V^2)$



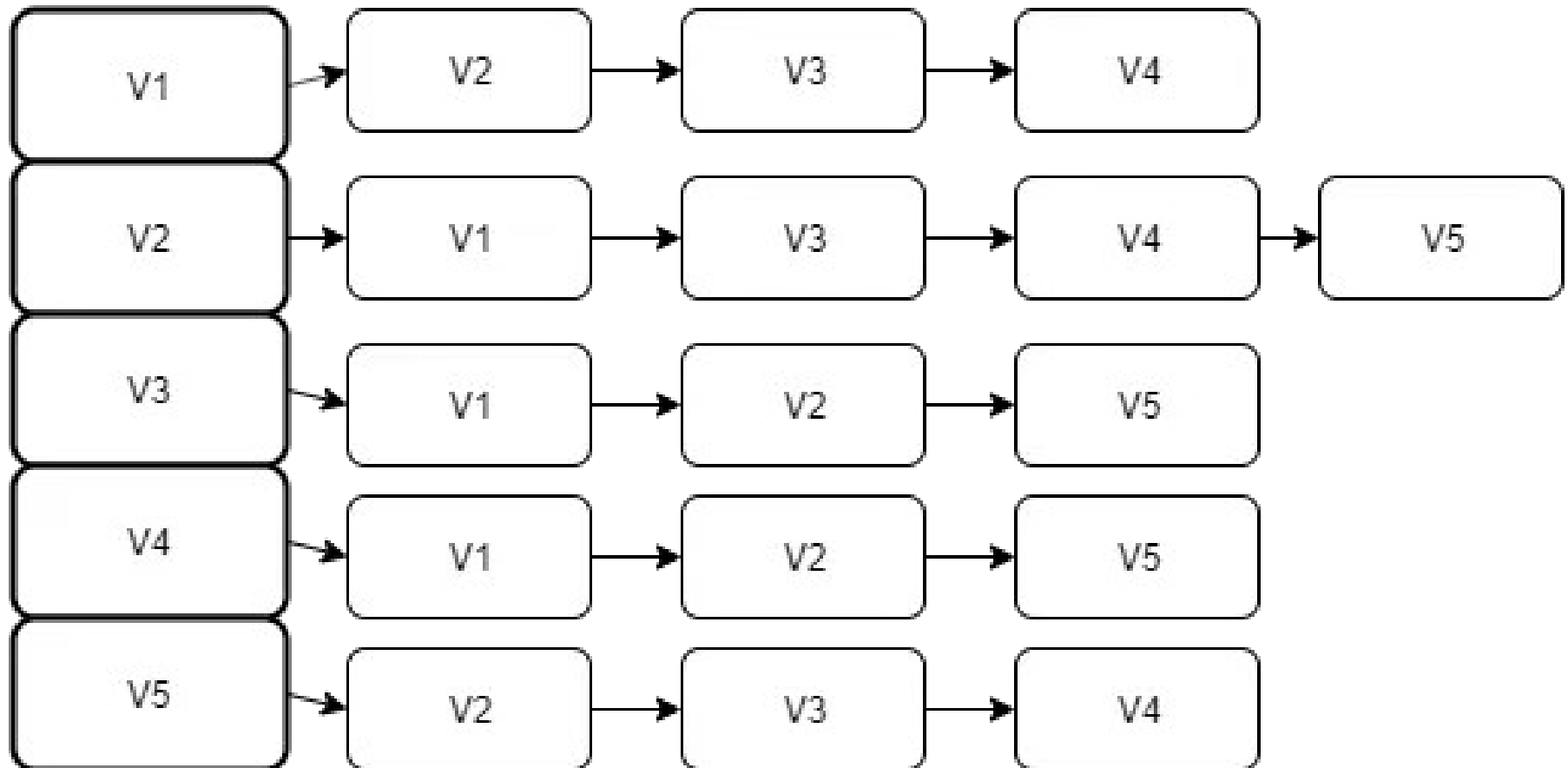
	V1	V2	V3	V4	V5
V1	0*	20	4	0*	23
V2	20	0*	17	58	0*
V3	4	17	3	3	15
V4	0*	58	3	0*	0*
V5	23	0*	15	0*	0*

*zero = nenhuma ligação

Lista de Adjacência $\Theta(V+E)$

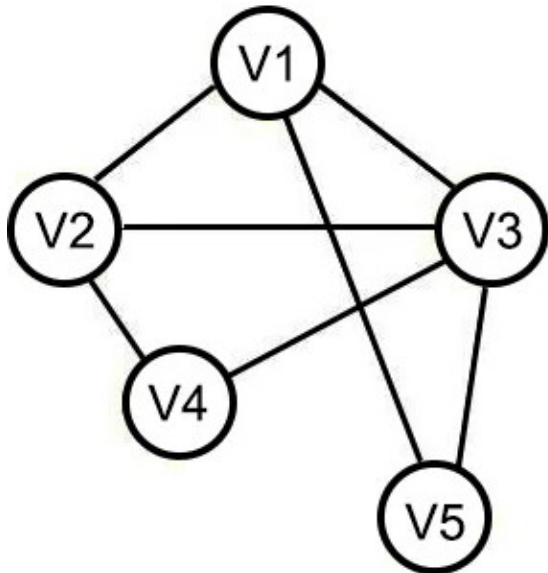


Grafo **não** direcionado



Grafos

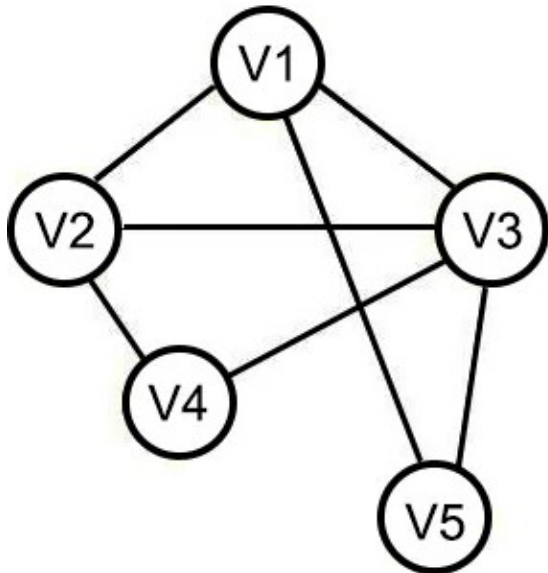
Grafo **não** direcionado



	V1	V2	V3	V4	V5
V1	0	1	1	0	1
V2	1	0	1	1	0
V3	1	1	0	1	1
V4	0	1	1	0	0
V5	1	0	1	0	0

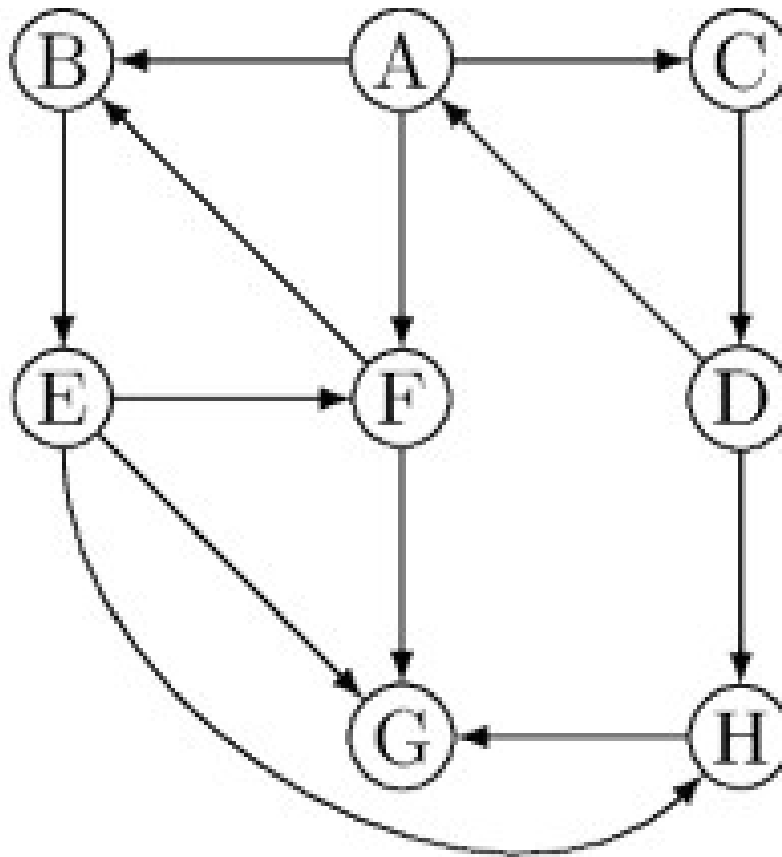
Grafos

Grafo **não** direcionado



	V1	V2	V3	V4	V5
V1	0	1	1	0	1
V2	1	0	1	1	0
V3	1	1	0	1	1
V4	0	1	1	0	0
V5	1	0	1	0	0

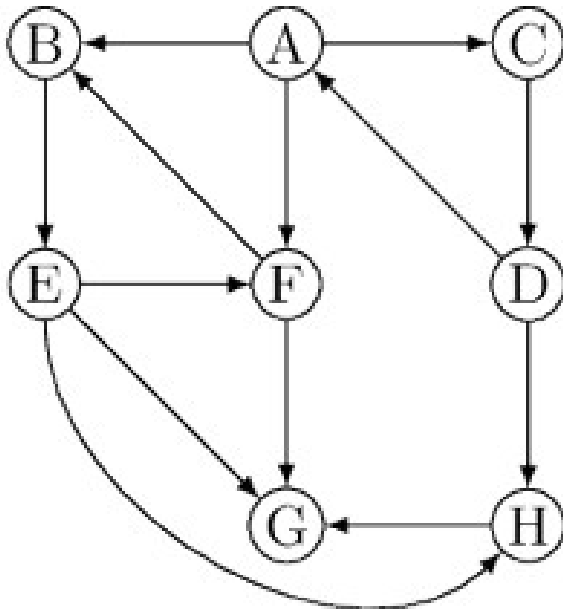
Grafo direcionado



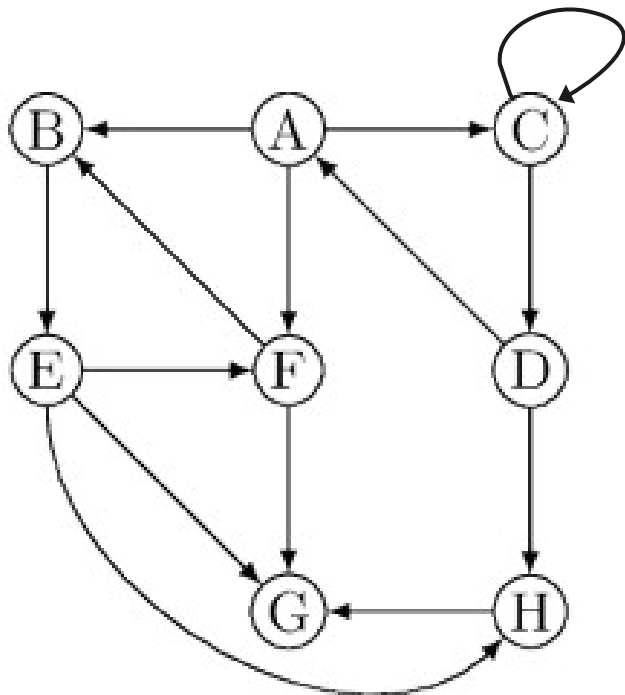
Grafos

Grau de um vértice: número de arestas que chegam ou partem dele.

- Grau de entrada: número de arestas que chegam no vértice
- Grau de saída: número de arestas que partem do vértice

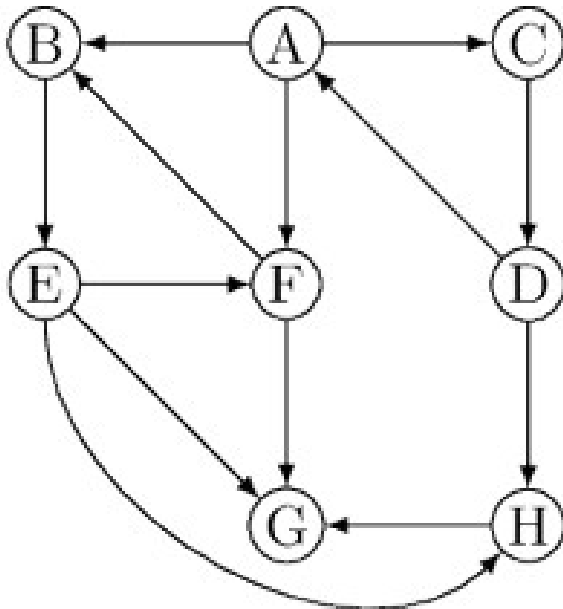


Laço: Uma aresta é chamada de laço se o vértice de partida é o vértice de entrada.



Caminho: sequência de vértices conectados a uma estrutura linear, sem ciclos (nenhum vértice se repete).

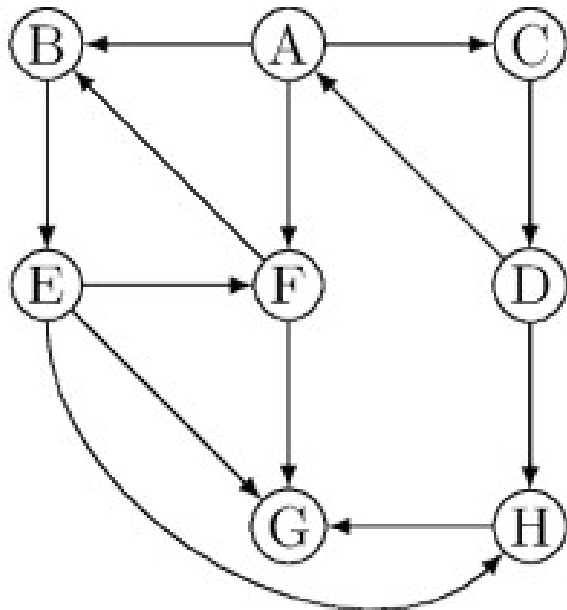
- Comprimento do caminho: número de arestas do caminho



Depth First-Search

(*Busca em Profundidade*)

O **algoritmo de Busca em Profundidade** realiza uma busca ou travessia em um grafo. É utilizado para verificar quais vértices são acessíveis iniciando um caminho a partir de um vértice específico



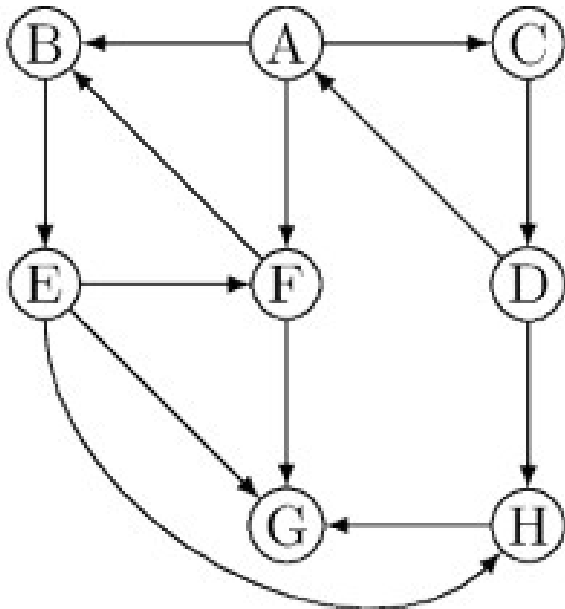
Quais vértices são acessíveis partindo de A?

Quais vértices são acessíveis partindo de E?

Depth First-Search

(*Busca em Profundidade*)

O **algoritmo de Busca em Profundidade** realiza uma busca ou travessia em um grafo. É utilizado para verificar quais vértices são acessíveis iniciando um caminho a partir de um vértice específico



```
// G = grafo, s = vértice inicial,  
// v = vetor de visitação de  
vértices
```

```
DFS( G, s, v )
```

```
    v[s] = true
```

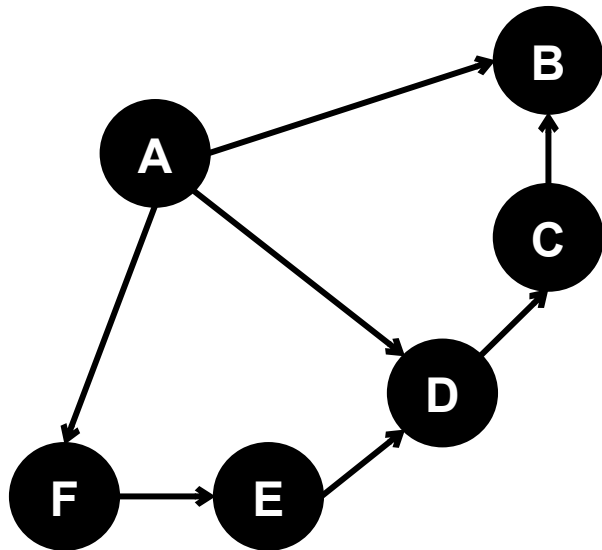
```
    para cada u ∈ G.adj[s]
```

```
        if( v[u] == false )
```

```
            DFS( G, u, v )
```

Depth First-Search

(*Busca em Profundidade*)



Complexidade de tempo:

Matriz / Lista

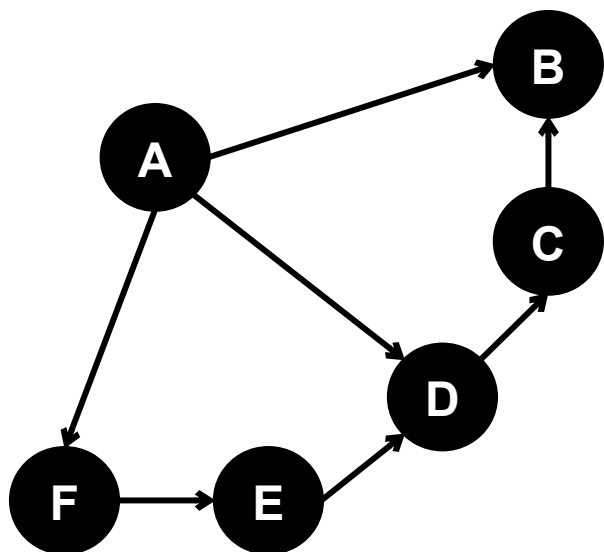
Melhor caso:

Pior caso:

Complexidade de espaço:

Depth First-Search

(*Busca em Profundidade*)



Complexidade de tempo:

Matriz

Melhor caso: $\Omega (V)$

Nenhum nó está acessível a partir de um determinado nó

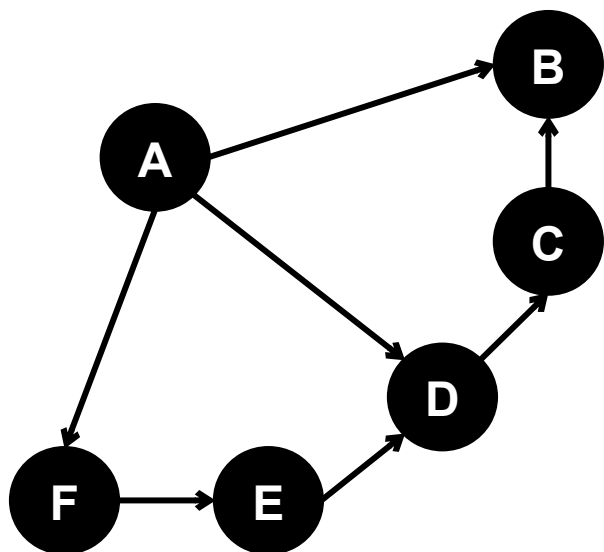
Pior caso: $O (V^2)$

Todos os outros nós estão acessíveis a partir de um nó

Complexidade de espaço: precisamos manter o vetor de nós visitados $\Theta (V)$

Depth First-Search

(*Busca em Profundidade*)



Complexidade de tempo:

Lista

Melhor caso: $\Omega (V)$

Necessidade de inicializar
vértices como não visitados

Pior caso: $O (V + E)$

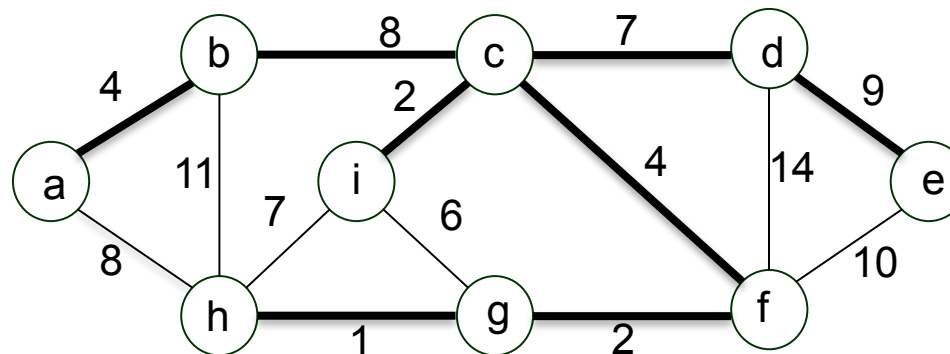
Todos os outros nós estão
acessíveis a partir de um nó

Complexidade de espaço: precisamos manter o vetor
de nós visitados $\Theta (V)$

Prim

(Árvore Geradora Mínima)

Uma **árvore geradora** para um grafo conexo é uma árvore que conecta todos os vértices do grafo e que o conjunto de arestas é um subconjunto das arestas desse grafo. A árvore geradora é mínima se o somatório dos custos associados cada arestas for o menor possível.

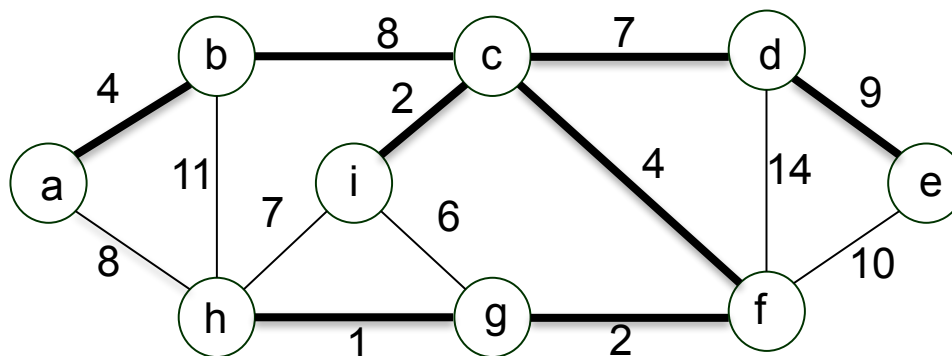


Prim

(Árvore Geradora Mínima)

0 Prim funciona “expandindo” uma árvore aos poucos:

1. Escolhe um vértice inicial qualquer.
2. Entre todas as arestas que ligam:
 - um vértice já visitado
 - a um vértice não visitado,escolhe a aresta de menor peso.
3. Adiciona essa aresta e o novo vértice à árvore.
4. Repete até visitar todos os vértices.



Prim

(Árvore Geradora Mínima)

// G = grafo, s = vértice inicial, w = vetor de custo

MST-PRIM(G, s, w)

para cada u $\in$ G.V

w[u] = ∞

u.pai = null

w[s] = 0

q.add({0,s})

enquanto q $\neq \emptyset$

u = extrai-minimo(q)

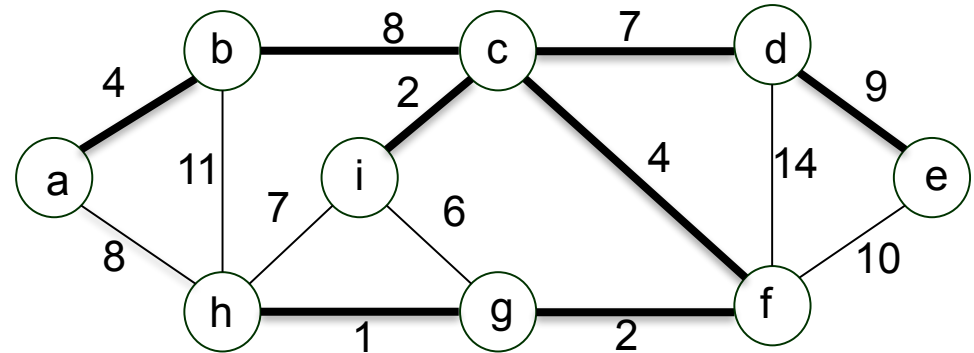
para cada z $\in$ G.adj[u]

se z.custo < w[z.dest]

q.add({z.custo, z.dest}) // Qual a estrutura??

w[z.dest] = z.custo

z.dest.pai = u



// |V| vezes

// Qual a estrutura usada??

// Qual a estrutura usada??

// Qual a estrutura??

Prim

(Árvore Geradora Mínima)

// G = grafo, s = vértice inicial, w = vetor de custo

MST-PRIM(G, s, w)

para cada u $\in$ G.V

 w[u] = ∞

 u.pai = null

 w[s] = 0

 q.add({0,s})

Usando G=>Matriz e q=>Lista

Tempo:

$O(|V|^2 + |V|^2)$

$O(|V|^2)$

enquanto q $\neq \emptyset$

// |V| vezes

 u = extrai-minimo(q)

// |V| verificações

para cada z $\in$ G.adj[u]

// |V| vezes

se z.custo < w[z.dest]

 q.add({z.custo, z.dest}) // 0(1)

 w[z.dest] = z.custo

 z.dest.pai = u

Prim

(Árvore Geradora Mínima)

// G = grafo, s = vértice inicial, w = vetor de custo

MST-PRIM(G, s, w)

para cada u $\in$ G.V

 w[u] = ∞

 u.pai = null

 w[s] = 0

 q.add({0,s})

Usando $G \Rightarrow$ Lista e $q \Rightarrow$ Lista

Tempo:

$O(|V^2| + |E|)$

* lembrando que $E = O(V^2)$

enquanto q $\neq \emptyset$

// * { $O(V)$ }

 u = extrai-minimo(q)

// * $\Rightarrow O(V^2)$ verificações

para cada z $\in$ G.adj[u]

// * $\Rightarrow O(E)$

se z.custo < w[z.dest]

 q.add({z.custo, z.dest}) // $O(1)$

 w[z.dest] = z.custo

 z.dest.pai = u

Prim

(Árvore Geradora Mínima)

// G = grafo, s = vértice inicial, w = vetor de custo

MST-PRIM(G, s, w)

para cada u $\in$ G.V

 w[u] = ∞

 u.pai = null

 w[s] = 0

 q.add({0,s})

Usando G=>Lista e q=>Árvore

Tempo:

$O(|V \log V + E \log V|)$

$O((V+E) \log V)$

enquanto q $\neq \emptyset$

 u = extrai-minimo(q)

// * { $O(V)$ }

// * $\Rightarrow O(V \log V)$

para cada z $\in$ G.adj[u]

// * $\Rightarrow O(E)$

se z.custo < w[z.dest]

 q.add({z.custo, z.dest}) // (log V) [Se não

 w[z.dest] = z.custo

 manter duplicatas]

 z.dest.pai = u

Prim

(Árvore Geradora Mínima)

// G = grafo, s = vértice inicial, w = vetor de custo

MST-PRIM(G, s, w)

para cada u $\in$ G.V

 w[u] = ∞

 u.pai = null

 w[s] = 0

 q.add({0,s})

Usando G=>Lista e q=>Min-Heap
(Fila de prioridades)

Tempo:
 $O(E \log V)$

enquanto q $\neq \emptyset$

// * { $O(V)$ }

 u = extrai-minimo(q)

// * $\Rightarrow O(\log V)$

para cada z $\in$ G.adj[u]

// * $\Rightarrow O(E)$

se z.custo < w[z.dest]

 q.add({z.custo, z.dest}) // (1)

 w[z.dest] = z.custo

 z.dest.pai = u

Prim

(Árvore Geradora Mínima)

// G = grafo, s = vértice inicial, w = vetor de custo

MST-PRIM(G, s, w)

para cada u $\in$ G.V

 w[u] = ∞

 u.pai = null

w[s] = 0

q.add({0,s})

A complexidade depende da estrutura de dados utilizada para gerenciar as arestas. A melhor Implementação (Fila de Prioridade) é ideal para grafos esparsos, enquanto a implementação simples (Matriz) é preferível para grafos densos.

enquanto q $\neq \emptyset$

// |V| vezes

 u = extraí-minimo(q)

// Qual a estrutura usada??

para cada z $\in$ G.adj[u]

// Qual a estrutura usada??

se z.custo < w[z.dest]

 q.add({z.custo, z.dest}) // Qual a estrutura??

 w[z.dest] = z.custo

 z.dest.pai = u

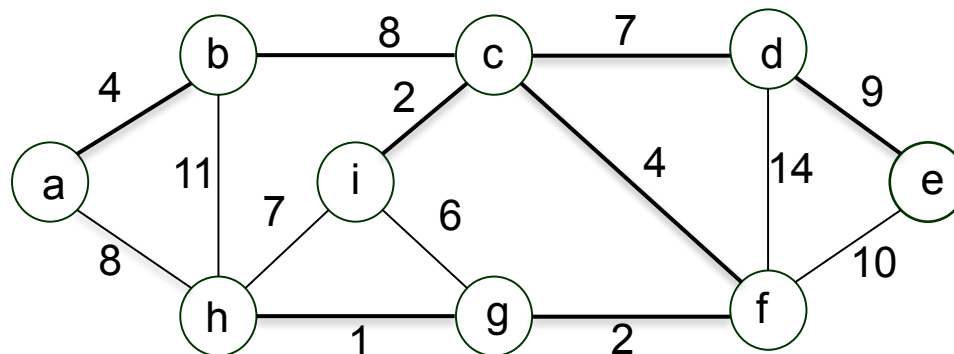
Dijkstra

(menor caminho entre dois nós)

Dado um grafo ponderado – **sem** arestas de peso negativo – e dois nós, o algoritmo de Dijkstra encontra o menor caminho entre estes dois nós.

Semelhante ao algoritmo de Prim:

1. escolhe o vértice não visitado com menor distância;
2. verificar se existe um caminho melhor (mais curto) passando pelo vértice atual.;
3. atualiza distâncias menores encontradas.



Dijkstra

(menor caminho entre dois nós)

// G = grafo, s = vértice inicial, w = vetor de custo

DIJKSTRA(G, s, w)

para cada u $\in$ G.V

 w[u] = ∞

 u.pai = null

 w[s] = 0

 q.add({0,s})

Usando G=>Lista e q=>Min. Heap

Tempo:

$O(E + V \log V) = O(E \log V)$

enquanto q $\neq \emptyset$

// * { $O(V)$ }

 u = extrai-minimo(q)

// * $\Rightarrow O(V \log V)$

para cada z $\in$ G.adj[u]

// * $\Rightarrow O(E)$

se w[u] + z.custo < w[z.dest]

 q.add({w[u] + z.custo, z.dest}) // (1)

 w[z.dest] = w[u] + z.custo

 z.dest.pai = u

A* (A-Estrela)

(menor caminho entre dois nós)

// G=grafo, s=vertice inicial, t=destino, w=vetor custo

DIJKSTRA(G, s, t, w)

para cada u $\in$ G.V

w[u] = ∞

f[u] = ∞

u.pai = null

w[s] = 0

q.add({0,s})

enquanto q $\neq \emptyset$

u = extrai-minimo(q)

se u = t **retorne**

para cada z $\in$ G.adj[u]

custoTemp = w[u] + z.custo

se custoTemp < w[z.dest]

w[z.dest] = custoTemp

f[z.dest] = w[z.dest] + h(z.dest, t)

q.add({f[z.dest], z.dest}) // (1)

z.dest.pai = u

Usando G=>Lista e q=>Min. Heap

Tempo:

$O(E \log V)$

Mais eficiente que o Dijkstra,
mas pode achar o melhor, mas
sim achar um dos melhores dos
melhores

A* (A-Estrela)

(menor caminho entre dois nós)

// G=grafo, s=vertice inicial, t=destino, w=vetor custo

DIJKSTRA(G, s, t, w)

para cada u $\in$ G.V

 w[u] = ∞

 f[u] = ∞

 u.pai = null

 w[s] = 0

 q.add({0,s})

enquanto q $\neq \emptyset$

// * { 0(V) }

 u = extrai-minimo(q)

// * $\Rightarrow 0(V \log V)$

 se u = t retorne

// **parada antecipada**

para cada z $\in$ G.adj[u]

// * $\Rightarrow 0(E)$

 custoTemp = w[u] + z.custo

se custoTemp < w[z.dest]

 w[z.dest] = custoTemp

 f[z.dest] = w[z.dest] + h(z.dest, t)

 q.add({f[z.dest], z.dest}) // (1)

 z.dest.pai = u

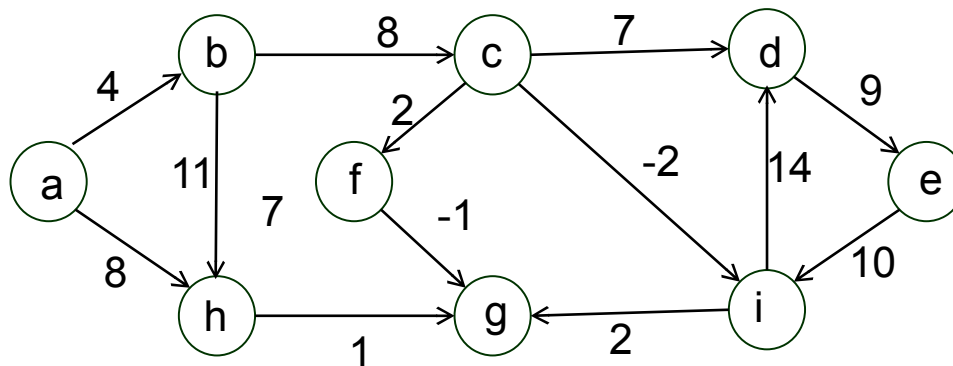
O A* não é intrinsecamente mais eficiente no pior caso — ele é mais eficiente na prática porque usa informação extra para evitar explorar partes inúteis do grafo.

Bellman Ford

(menor caminho entre dois nós)

Dado um grafo ponderado – possivelmente **com arestas de peso negativo**, mas sem ciclo – e dois nós, o algoritmo de Belmann Ford encontra o menor caminho entre os dois nós.

Menor Caminho entre b | g

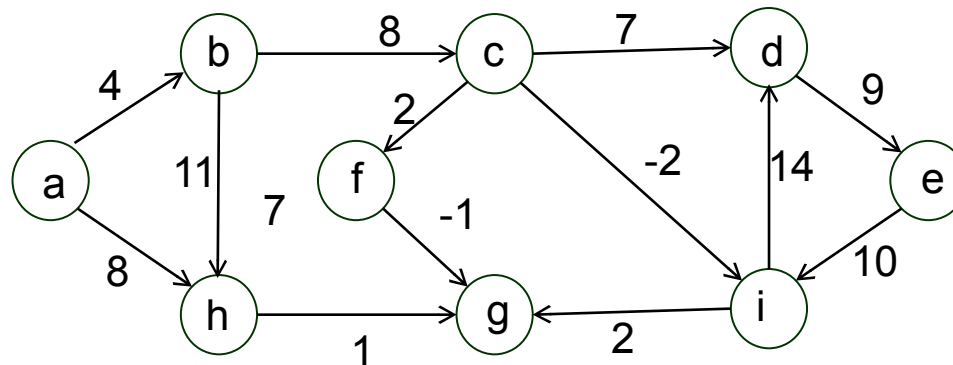


Bellman Ford

(menor caminho entre dois nós)

1. Para cada $V-1$:
2. para cada aresta (u,v)
3. Verifica se consegue melhorar algo, existe ciclo negativo

Menor Caminho entre b | g



"Vou tentar melhorar todos os caminhos de novo e de novo até estabilizar."

Bellman Ford

(menor caminho entre dois nós)

// G = grafo, s = vértice inicial, w = vetor de custo

BELLMAN(G, s, w)

para cada u ∈ G.V

 w[u] = ∞

 u.pai = null

 w[s] = 0;

para i de 0 a size(vertices):

para cada u ∈ G.V:

para cada a ∈ G.adj[u]:

se (w[u] + a.custo < w[a.dest])

 w[a.dest] = w[u] + a.custo

 a.dest.pai = u

Usando G=>Lista

Tempo:

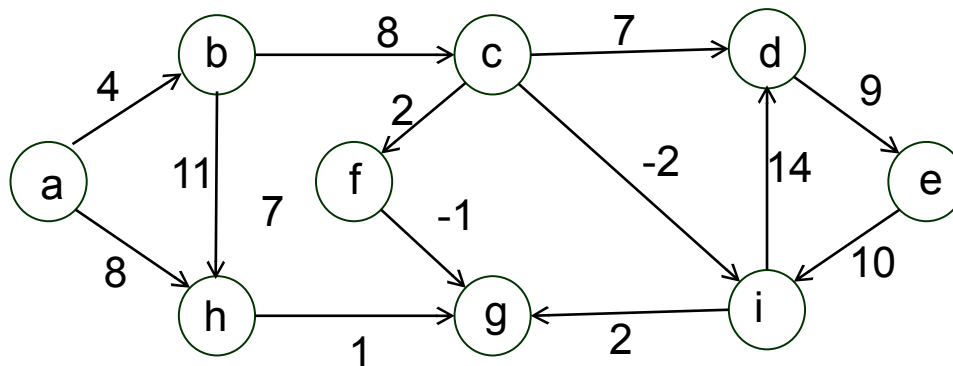
$O(V + V \cdot E) = O(V \cdot E)$

Repete E vezes

Floyd Warshall

(menor caminho entre todos os nós)

Dado um grafo ponderado – possivelmente com arestas de peso negativo, mas sem ciclo – o algoritmo de Floyd Warshall encontra o menor caminho entre todos os nós (de todos para todos).



Floyd Warshall

(menor caminho entre todos os nós)

```
// G = grafo
```

```
FLOYD( G )
```

```
  dist[G.v][G.v]
```

```
  para cada u ∈ G.v
```

```
    dist[u][u] = 0
```

```
  para cada z(u,v) ∈ G.E
```

```
    dist[u][v] = z.custo
```

```
  para k de 0 a G.v:
```

```
    para i de 0 a G.v:
```

```
      para j de 0 a G.v:
```

```
        se ( dist[i][j] > dist[i][k] + dist[k][j] )
```

```
          dist[i][j] = dist[i][k] + dist[k][j]
```

Usando G=>Lista / Matriz

Tempo:

$\Theta (V^3)$